

Lab 06:

Question#01:

CODE:

```
#include <iostream>
using namespace std;

class Shape
{
protected:
    int numberOfSides;
    double area;

public:
    Shape(int sides, double area = 0) : numberOfSides(sides), area(area)
    {
    }
    int getNumberOfSides()
    {
        return numberOfSides;
    }
    int getArea()
    {
        return area;
    }

    void setNumberOfSides(int sides)
    {
        numberOfSides = sides;
    }
    void setArea(double a)
    {
        area = a;
    }

    virtual void generateArea() = 0; // Virtual Function so derived classes can
// pure virtual --> Shape is abstract.
};

class Rectangle : public Shape
```

Object Oriented Programming:

```
{
protected:
    double length, width;

public:
    Rectangle(double l, double w) : Shape(4), length(l), width(w)
    {
    }

    void generateArea() override
    {
        area = length * width;
    }

    double getLenght()
    {
        return length;
    }
    double getWidth()
    {
        return width;
    }
};

class Circle : public Shape
{
private:
    double radius;

public:
    Circle(double r) : Shape(0), radius(r)
    {
    }
    void generateArea() override
    {
        area = 3.142 * radius * radius;
    }
};

class Triangle : public Shape
{
private:
    double height, base;

public:
```

Object Oriented Programming:

```
Triangle(double h, double b) : Shape(3), height(h), base(b)
{
}
void generateArea() override
{
    area = (height * base) / 2;
}
};

class Square : public Rectangle
{
public:
    Square(double side) : Rectangle(side, side)
    {
    }

    void checkIfSquare()
    {
        if (length == width)
        {
            cout << "This is a square." << endl;
        }
        else
        {
            cout << "This is not a square." << endl;
        }
    }

    void generateArea() override
    {
        area = length * width;
    }
};

int main()
{
    Rectangle rect(5, 3);
    rect.generateArea();
    cout << "Area of Rectangle: " << rect.getArea() << endl;

    Circle circ(4);
    circ.generateArea();
    cout << "Area of Circle: " << circ.getArea() << endl;

    Triangle tri(4, 6);
```

Object Oriented Programming:

```
tri.generateArea();  
cout << "Area of Triangle: " << tri.getArea() << endl;  
  
Square sq(4);  
  
sq.generateArea();  
cout << "Area of Square: " << sq.getArea() << endl;  
sq.checkIfSquare();  
return 0;  
}
```

Output:

```
Area of Rectangle: 15  
Area of Circle: 50  
Area of Triangle: 12  
Area of Square: 16  
This is a square.  
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 06> |
```

Question#02:

CODE:

```
#include <iostream>  
using namespace std;  
  
class Calculator{  
private:  
    double num1, num2, num3;  
  
public:  
    Calculator(int a, int b, int c) : num1(a), num2(b), num3(c){  
  
    }  
    double add(double a, double b){  
        return a + b;  
    }  
  
    double add(double a, double b, double c){  
        return a + b + c;  
    }  
}
```

Object Oriented Programming:

```
    }  
    double subtract(double a, double b){  
        return a - b;  
    }  
  
    double subtract(double a, double b, double c){  
        return a - b - c;  
    }  
  
    double multiply(double a, double b){  
        return a * b;  
    }  
    double multiply(double a, double b, double c){  
        return a * b * c;  
    }  
    double divide(double a, double b){  
        if(b == 0){  
            cout << "Division not possible" << endl; return 0;  
        }  
        return a / b;  
    }  
    double divide(double a, double b, double c){  
        if(b == 0 || c == 0){  
            cout << "Division not possible" << endl; return 0;  
        }  
        return a / b / c;  
    }  
};  
  
int main() {  
    Calculator calc(0, 0, 0);  
    double n1, n2, n3;  
    cout << "=====This is a basic arithmic calculator=====" << endl;  
    cout << "Enter 3 Numbers: " ;  
    cin >> n1 >> n2 >> n3;  
  
    cout << "Addition (2 Numbers): " << calc.add(n1, n2) << endl;  
    cout << "Addition (3 Numbers): " << calc.add(n1, n2, n3) << endl;  
    cout << "Subtraction (2 Numbers): " << calc.subtract(n1, n2) << endl;  
    cout << "Subtraction (3 Numbers): " << calc.subtract(n1, n2, n3) << endl;  
    cout << "Multiplicattion (2 Numbers): " << calc.multiply(n1, n2) << endl;  
    cout << "Multiplicattion (3 Numbers): " << calc.multiply(n1, n2, n3) << endl;
```

Object Oriented Programming:

```
    cout << "Division (2 Numbers): " << calc.divide(n1, n2) << endl;
    cout << "Division (3 Numbers): " << calc.divide(n1, n2, n3) << endl;
    return 0;
}
```

Output:

```
g (OOPs)\Labs\Lab 06\ ; if ($?) { g++ C1-25276_Q2.cpp -o C1-25276_Q2 } ; if ($?) { .\C1-25276_Q2 }
=====This is a basic arithmetic calculator=====
Enter 3 Numbers: 5 10 15
Addition (2 Numbers): 15
Addition (3 Numbers): 30
Subtraction (2 Numbers): -5
Subtraction (3 Numbers): -20
Multiplication (2 Numbers): 50
Multiplication (3 Numbers): 750
Division (2 Numbers): 0.5
Division (3 Numbers): 0.0333333
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 06> |
```

Question#03:

CODE:

```
#include <iostream>
#include <cmath>    // for sqrt() in magnitude
using namespace std;

class Vector {
private:
    double x, y;

public:
    // _____
    // Constructor
    // _____
    Vector(double x = 0, double y = 0) : x(x), y(y) {}

    // _____
    // Operator Overloading
    // _____

    // + operator: adds two Vectors
    Vector operator+(const Vector& other) {
```

Object Oriented Programming:

```
        return Vector(x + other.x, y + other.y);
    }

    // - operator: subtracts two Vectors
    Vector operator-(const Vector& other) {
        return Vector(x - other.x, y - other.y);
    }

    // * operator: multiplies Vector by a scalar
    Vector operator*(double scalar) {
        return Vector(x * scalar, y * scalar);
    }

    // / operator: divides Vector by a scalar
    Vector operator/(double scalar) {
        if (scalar == 0) {
            cout << "Error: Division by zero!" << endl;
            return Vector(0, 0);
        }
        return Vector(x / scalar, y / scalar);
    }

    // -----
    // Magnitude:  $\sqrt{x^2 + y^2}$ 
    // -----
    double magnitude() {
        return sqrt(x * x + y * y);
    }

    // -----
    // Display the vector nicely
    // -----
    void display(const string& label = "") {
        if (!label.empty()) cout << label << ": ";
        cout << "(" << x << ", " << y << ")" << endl;
    }
};

int main() {
    Vector v1(4, 3);
    Vector v2(1, 2);

    v1.display("v1");
    v2.display("v2");
}
```

Object Oriented Programming:

```
cout << "\n--- Operator Demonstrations ---" << endl;

// Addition
Vector sum = v1 + v2;
sum.display("v1 + v2");

// Subtraction
Vector diff = v1 - v2;
diff.display("v1 - v2");

// Multiplication by scalar
Vector scaled = v1 * 3;
scaled.display("v1 * 3");

// Division by scalar
Vector divided = v1 / 2;
divided.display("v1 / 2");

// Magnitude
cout << "\nMagnitude of v1: " << v1.magnitude() << endl; //  $\sqrt{16+9} = 5$ 
cout << "Magnitude of v2: " << v2.magnitude() << endl; //  $\sqrt{1+4} = 2.236$ 

return 0;
}
```

Output:

```
v1: (4, 3)
v2: (1, 2)

--- Operator Demonstrations ---
v1 + v2: (5, 5)
v1 - v2: (3, 1)
v1 * 3: (12, 9)
v1 / 2: (2, 1.5)

Magnitude of v1: 5
Magnitude of v2: 2.23607
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 06> |
```

Question#03:

CODE:

Object Oriented Programming:

```
#include <iostream>
using namespace std;
// Base class Person
class Person {
protected:
    string name;
    string occupation;

public:
    Person(string occ) {
        occupation = occ;
    }

    virtual void Draw() {
        cout << "A person can draw in many ways" << endl;
    }
};

// Derived class Artist
class Artist : public Person {
public:
    Artist() : Person("artist") {

    }

    void Draw() override {
        cout << "An artist can draw with a paint brush" << endl;
    }
};

// Derived class Gunman
class Gunman : public Person {
public:
    Gunman() : Person("gunman") {

    }

    void Draw() override {
        cout << "A gunman draws a gun to shoot" << endl;
    }
};

int main() {
    // Array of Person pointers of size 3 (Person, Artist, Gunman)
    Person* arr[3];
}
```

Object Oriented Programming:

```
// Dynamically create objects
arr[0] = new Person("person");
arr[1] = new Artist();
arr[2] = new Gunman();

for (int i = 0; i < 3; i++) {
    arr[i]->Draw();
}

delete arr[0];
delete arr[1];
delete arr[2];

return 0;
}
```

Output:

```
A person can draw in many ways
An artist can draw with a paint brush
A gunman draws a gun to shoot
PS D:\NED University\2nd Semester\Object Oriented Programming (OOPs)\Labs\Lab 06> 
```

Made By: Faarid Amir Shaikh CT- 25276

Made By: Faarid Amir Shaikh CT-25276 : Section F

Object Oriented Programming: